

lezione100424

September 22, 2024

Laboratorio di WebScraping

Anno Accademico 2023-2024

Docente: Laura Ricci

Lezione 17

Selenium: Implicit e Explicit Wait

Crawling Sites

Bitcoin DataSet

10 Aprile 2024

0.1 Perché c'è bisogno di Wait in Selenium?

- uso di **AjaX** e **JAVAScript**
- gli elementi con cui vogliamo interagire sono caricati in momenti diversi
- se l'elemento non è ancora stato caricato, può essere sollevata **NoSuchElementException**

0.2 Selenium Wait Functions

- vi sono situazioni in cui è necessario **attendere** prima di interagire con gli elementi della pagina
- le operazioni di **wait** consentono di **attendere** che un elemento sia **caricato**, prima di eseguire una qualsiasi azione su tale elemento
- **Selenium** offre il package **Wait** che include tre tipi di **wait**
 - **Explicit Waits**
 - **Implicit Waits**
 - **Fluent Waits**
- possibile utilizzare anche la funzione **sleep** di **Python**
 - aspetta per un intervallo di tempo predeterminato
 - non consigliato perchè imposta un tempo assoluto, non collegato al caricamento degli elementi

0.3 Selenium Implicit Wait

- impostano un **tempo di attesa globale**
- quando si ricerca un elemento nel **DOM**, e l'elemento non esiste, il WebDriver interroga la pagina ripetutamente per un certo periodo di tempo
- lo stesso tempo di attesa per **ogni elemento**
 - una sorta di **catch all**, un tempo valido per tutti gli elementi
- differenza con **time.sleep()**
 - non appena l'elemento è caricato, prosegue con l'esecuzione, invece la **sleep** attende comunque l'intervallo di tempo prestabilito
 - se passa l'intervallo di tempo stabilito, viene sollevata una **NoSuchElementException**
- applicato a tutti i successivi comandi del **WebDriver** e valido per l'intera sessione, **per default vale 0**
- la sintassi è la seguente

```
from selenium import webdriver driver = webdriver.Chrome() driver.implicitly_wait(10)
# seconds driver.get("http://somedomain/url_that_delays_loading") myDynamicElement =
driver.find_element_by_id("myDynamicElement")
```

0.4 Selenium Explicit Wait: Expected Conditions

- applicabile a **elementi specifici**, con **condizioni specifiche**, ad esempio il fatto che uno specifico elemento diventi visibile o cliccabile

```
from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.wait import WebDriverWait from selenium.webdriver.support import expected_conditions as EC
```

```
driver = webdriver.Firefox() driver.get("http://somedomain/url_that_delays_loading") try: element = WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, "myDynamicElement"))) ) finally: driver.quit()
```

- il **WebDriver** aspetta al massimo **10 secondi** che l'elemento individuato sia presente nel **DOM**

0.5 Selenium Explicit Wait: Expected Conditions

- **element_to_be_clickable(locator)**: aspetta che l'elemento sia visibile e abilitato a ricevere un click
- **visibility_of_element_located(locator)**: aspetta che l'elemento sia presente nel **DOM** e visibile
- **presence_of_element_located(locator)**: aspetta che l'elemento sia visibile nel **DOM**
- **title_contains(title)**: aspetta che il titolo della pagina contenga un testo specifico
- **text_to_be_present_in_element(locator, text)**: Aspetta che uno specifico testo sia presente in un elemento specifico
- **invisibility_of_element_located(locator)**: aspetta che un elemento diventi invisibile
-

0.6 Selenium Fluent Wait

- un tipo di **Explicit Wait** che fornisce ulteriori parametri di configurazione
- particolarmente utile quando
 - si vuole controllare la condizione a intervalli regolari
 - si vogliono ignorare certi tipi di eccezioni durante il periodo di wait

```
from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.support.wait import WebDriverWait from selenium.webdriver.support import expected_conditions as EC
```

```
driver = webdriver.Firefox() driver.get("http://somedomain/url_that_delays_loading") try: element = WebDriverWait(driver, timeout=30, poll_frequency=5, ignored_exceptions= [NoSuchElementException]). until(EC.presence_of_element_located((By.ID, "myDynamicElement"))) finally: driver.quit()
```

0.7 Scraping IATA

- <http://www.iata.org/publications/Pages/code-search.aspx>
- un motore di ricerca per la ricerca di
 - **compagnie aeree**, dato il codice di 2 lettere della compagnia restituisce il nome e viceversa
 - **aeroporti**, dato il codice di 3 lettere dell'aeroporto, ne restituisce il nome e viceversa
- elementi importanti per la risoluzione dell'esercizio
 - il sito manda un **pop up We use cookies to give you the best experience on our website. We also use cookies for advertising purposes. Please see our privacy policy and cookies policy for complete information.** ** occorre cliccare il bottone **Ok, I got it**
 - inviare i dati in una form di ricerca
 - appare una tabella di dati come risposta
 - vogliamo inserire i dati ricevuti in un DataFrame **PANDAS**

0.8 Scraping IATA

0.9 Scraping IATA

0.10 Scraping IATA

0.11 Scraping IATA: Implicit wait

```
[ ]: import pandas
from selenium import webdriver
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
import time

url = 'https://www.iata.org/en/publications/directories/code-search/'

driver = webdriver.Chrome()
driver.implicitly_wait(10)
```

0.12 Scraping IATA: Implicit wait

```
[ ]: def get_results(airline_name):
    driver.get(url)
    time.sleep(10)
    element = driver.find_element(By.CSS_SELECTOR, "body > div.gdpr-banner > div_
    ↪ div.gdpr-button-wrapper > button")
    print(element)
    element.click()
    driver.find_element(By.NAME, "airline.search").send_keys(airline_name)
    button=driver.find_element(By.XPATH, "/html/body/div[1]/div[2]/div/div/
    ↪div[2]/main/div[3]/div[1]/div/form/div[2]/div[1]/div[2]/button")
    button.click()
    print("cliccato!")
    print("*****")

    button.click()
    element = (By.CLASS_NAME, "datatable")
    table=driver.find_element(By.CLASS_NAME, "datatable")
    table_html = table.get_attribute('outerHTML')
    df = pandas.read_html(str(table_html))
    return df
```

0.13 Scraping IATA: Implicit wait

```
[ ]: df = get_results('Ryanair')
print("****")
print(df)
driver.quit()
```

0.14 Scraping IATA: Explicit wait

```
[ ]: import pandas
from selenium import webdriver
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
url = 'https://www.iata.org/en/publications/directories/code-search/'
driver = webdriver.Chrome()
```

0.15 Scraping IATA: Explicit wait

```
[ ]: def get_results(airline_name):
    driver.get(url)
    try:
        element = WebDriverWait(driver, 10).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, "body > div.gdpr-banner_
↪ div > div.gdpr-button-wrapper > button")))
    )
    except:
        driver.quit()
    element.click()
    driver.find_element(By.NAME, "airline.search").send_keys(airline_name)
    button=driver.find_element(By.XPATH, "/html/body/div[1]/div[2]/div/div/
↪div[2]/main/div[3]/div[1]/div/form/div[2]/div[1]/div[2]/button")
    button.click()
    print("cliccato!")
    print("*****")

    button.click()
    table = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.CLASS_NAME, "datatable")))
    table_html = table.get_attribute('outerHTML')
    df = pandas.read_html(str(table_html))
    return df
```

0.16 Scraping IATA: Explicit wait

```
[ ]: df = get_results('Ryanair')
print("****")
print(df)
driver.quit()
```

0.17 Web Scraping o Web Crawling?

- i termini **Web Scraping** e **Web Crawling** sono spesso utilizzati in modo intercambiabile e non preciso

- in effetti, le due tecniche condividono alcune caratteristiche, ma sono diverse
- **Web Scraping**
 - estrazione di dati da una pagina web o anche da più pagine di un sito web
 - generalmente si individua a priori un insieme di pagine da cui si vuole estrarre informazione, per poi analizzarla
 - esempio
 - * una azienda vuole fare ricerche di mercato per lanciare un nuovo prodotto, ad esempio un nuovo laptop
 - * estrae da **Amazon** la lista dei laptop sul mercato con il relativo prezzo, per capire come posizionarsi sul mercato
 - * non è limitato ad una singola pagina, ma comunque viene effettuato su un insieme di pagine predeterminato
- **Web crawling**
 - utilizza **bots** o **spiders** per leggere tutto il contenuto di un sito web e/o di più siti web
 - in genere utilizzato per indicizzare il contenuto di un sito web, dai search engines (**Google, Bing,...**)
 - * iniziare da una **parola chiave** inserita in un motore di ricerca
 - * crawling di tutte le pagine restituite per quella parola chiave
 - * collezione di immagini, liste, tabelle da tutte le pagine
 - * navigare tra domini diversi, sul web

0.18 Web Crawling: metodologie

- preferibile usare un database per salvare i dati: molto spesso la varietà e la quantità dei dati richiedono l'uso di un database vero e proprio, piuttosto che un file **.csv**, oppure **JSON**
- strutturare il programma in modo da separare la parte di **crawling** dalla parte di **scraping**
 - **crawling**
 - * visita dei siti, estrazione di links, inserimento dei link in una coda per essere considerati sequenzialmente
 - **scraping**
 - * estrarre l'informazione di interesse dalle pagine reperite
- stabilire una condizione di **terminazione** del processo di **crawling**
 - durante il processo di crawling si visitano pagine e si collezionano altri link da visitare
 - quale condizione di terminazione se l'insieme delle pagine non è predeterminato?
 - inserire nuovi link in una coda e stabilire una condizione di fermata
- usare **parallelismo/concorrenza**
 - strutturare l'applicazione in modo che possa attivare più istanze di **crawler** in parallelo, che seguono percorsi diversi.
- tenere sempre presenti gli **aspetti legali!!**

0.19 DataSet: a “data base for lazy people”

- <https://dataset.readthedocs.io/en/latest/index.html#>
- può essere installata con **pip install**
- può usare come backend diversi database relazionali
 - **MySQL**

- Postgres
- SQLite
- useremo come back end **SQLite**
 - lightweight, serverless, self-contained data-base
 - integrato in Python: non occorrono installazioni aggiuntive
 - adatto per progetti di piccole dimensioni
- **DataSet** fornisce una semplice interfaccia verso **SQLite**, con alcune limitazioni

0.20 DataSet: a data base for lazy people

```
[ ]: import dataset
db = dataset.connect('sqlite:///mydatabase.db')
table = db['simpletable']

# Inserzione di un nuovo record
table.insert(dict(name='Mario Rossi', age=46, country='Italy'))

# dataset will create "missing" columns any time you insert a dict with an
↳ unknown key
table.insert(dict(name='Bob White', age=37, country='USA', gender='male'))

table.update(dict(name='Bob White', age=47), ['name'])
```

0.21 DataSet: a data base for lazy people

```
[ ]: print(db.tables)
```

```
[ ]: print(table.columns)
```

```
[ ]: print(db['simpletable'].columns)
```

```
[ ]: for user in db['simpletable']:
    print(user['age'])
```

0.22 DataSet: a data base for lazy people: queries

```
[ ]: result = db.query('SELECT country, COUNT(*) c FROM SIMPLETABLE GROUP BY
↳ country')
for row in result:
    print(row['country'], row['c'])
```

0.23 Crawling in parallel: la libreria JobLib

- Python offre diverse opzioni per la programmazione parallela
- adatta soprattutto per **embarassingly parallel computations**

- un programma che richiede di effettuare diverse computazioni
- ogni computazione è indipendente dalle altre
- ad esempio: calcolare la radice quadrata di tutti i numeri contenuti in una lista
- possibilità di eseguire le computazioni in parallelo, su core diversi

0.24 Crawling in parallel: la libreria JobLib

```
def square(x): time.sleep(1) # Simulating a time-consuming task return x ** 2
```

```
delayed_calls = [delayed(square)(number) for number in numbers] results_delayed = Parallel(n_jobs=-1)(delayed_calls)
```

0.25 Joblib: embarrassingly parallelism in Python

```
[15]: from joblib import Parallel, delayed
import time

def square(x):
    time.sleep(1) # Simulating a time-consuming task
    return x ** 2

# List of numbers
numbers = [1, 2, 3, 4, 5]

# Without using delayed
start = time.time()
results_no_delayed = [square(number) for number in numbers]
end = time.time()
time_no_delayed = end - start

# Using delayed
start = time.time()
delayed_calls = [delayed(square)(number) for number in numbers]
results_delayed = Parallel(n_jobs=-1)(delayed_calls)
end = time.time()
time_delayed = end - start

print("Results without delayed:", results_no_delayed)
print("Results with delayed: ", results_delayed)
print("Time without delayed: ", time_no_delayed)
print("Time with delayed: ", time_delayed)
```

```
Results without delayed: [1, 4, 9, 16, 25]
Results with delayed:    [1, 4, 9, 16, 25]
Time without delayed:    5.047278881072998
Time with delayed:       1.3561761379241943
```


0.26 Crawling and analysing Wikiperdia graph

- scopo dell'esercizio
 - individuare, mediante scraping i titoli delle pagine di wikipedia
 - individuare i link esistenti tra le pagine
 - costruire un grafo che rappresenti la relazione tra le pagine
 - analizzare il grafo
- cosa utilizzeremo
 - **BeutifulSoup**
 - la libreria **dataset** per creare un semplice database per memorizzare le informazioni reperite
 - **NetworkX** per analizzare e visualizzare il grafo risultante
 - la libreria **parallel**

0.27 Crawling and analysing Wikiperdia graph

0.28 Crawling and analysing Wikiperdia graph

- non vogliamo considerare link che puntano a siti esterni a Wikipedia
- ad esempio, nella figura sotto, i link che puntano a progetti “gemelli”, come **wikisource**

0.29 Crawling and analysing Wikiperdia graph

```
[ ]: import requests
import dataset
from bs4 import BeautifulSoup
from urllib.parse import urljoin, urldefrag
from joblib import Parallel, delayed

db = dataset.connect('sqlite:///wikipedia.db')
table_links = db['links']
table_pages = db['pages']
base_url = 'https://en.wikipedia.org/wiki/'

def store_page(url, title):
    print('Visited page:', url)
    print(' title:', title)
    table_pages.upsert({'url': url, 'title': title}, ['url'])

def store_links(from_url, links):
    db.begin()
    for to_url in links:
        table_links.upsert({'from_url': from_url, 'to_url': to_url},
↵ ['from_url', 'to_url'])
    db.commit()
    print("**link stored**")
```

0.30 Crawling and analysing Wikiperdia graph

```
[ ]: def get_random_unvisited_pages(amount=10):
    result = db.query('''SELECT * FROM LINKS
        WHERE to_url NOT IN (SELECT url FROM PAGES)
        ORDER BY RANDOM() LIMIT {}'''.format(amount))
    print( result)
    return [r['to_url'] for r in result]
```

- query **SQL-like** sul database
- si seleziona un link che porta ad una pagina che non è stata ancora visitata
 - si selezionano i links presenti nella tabella **LINKS**
 - si analizza il campo **to_url**, che indica la pagina a cui punta quel link
 - quella pagina non deve essere nelle **url** contenute nella tabella **PAGES**, che contiene le pagine già visitate

0.31 Crawling and analysing Wikiperdia graph

```
[ ]: def should_visit(base_url, url):
    if url is None:
        return None
    if not url.startswith(base_url):
        # questa è una URL esterna, non punta a una pagina interna di Wikiperia
        return None
    full_url = urljoin(base_url, url)
    full_url = urldefrag(full_url)[0]
    ignore = ['Wikipedia:', 'Template:', 'File:', 'Talk:', 'Special:',
        'Template talk:', 'Portal:', 'Help:', 'Category:', 'index.php']
    if any([i in full_url for i in ignore]):
        # Pagina da ignorare
        return None
    return full_url
```

0.32 Crawling and analysing Wikiperdia graph

```
[ ]: def get_title_and_links(base_url, url):
    html = requests.get(url).text
    html_soup = BeautifulSoup(html, 'html.parser')
    page_title = html_soup.find(class_='mw-page-title-main')
    page_title = page_title.text if page_title else ''
    links = []
    for link in html_soup.find_all("a"):
        link_url = should_visit(base_url, link.get('href'))
        if link_url:
            links.append(link_url)
    return url, page_title, links
```

- **Parallel**: classe offerta dal package **Joblib**
- il parametro **n_jobs** indica quanti processi eseguire in parallelo
 - valore **-1** indica che si vogliono usare tutti i job disponibili
- **delayed** è un **decorator** che permette alla classe **Parallel** di interagire con la funzione da eseguire

0.33 Crawling and analysing Wikiperdia graph

```
[ ]: if __name__ == '__main__':
    urls_to_visit = [base_url]
    print(urls_to_visit)
    while urls_to_visit:
        scraped_results = Parallel(n_jobs=5, backend="threading")(
            delayed(get_title_and_links)(base_url, url) for url in urls_to_visit
        )
        print("I am here")
        for url, page_title, links in scraped_results:
            store_page(url, page_title)
            store_links(url, links)
        urls_to_visit = get_random_unvisited_pages()
```

- prima soluzione per terminare il crawling: **interrompere il kernel nel Notebook**

0.34 Crawling and analysing Wikiperdia graph: analysis

```
[ ]: import networkx
import matplotlib.pyplot as plt
import dataset
db = dataset.connect('sqlite:///wikipedia.db')
G = networkx.DiGraph()
print('Building graph...')
for page in db['pages'].all():
    G.add_node(page['url'], title=page['title'])
for link in db['links'].all():
    # Only addedge if the endpoints have both been visited
    if G.has_node(link['from_url']) and G.has_node(link['to_url']):
        G.add_edge(link['from_url'], link['to_url'])
# Unclutter by removing unconnected nodes
G.remove_nodes_from(networkx.isolates(G))
```

0.35 Crawling and analysing Wikiperdia graph: analysis

```
[ ]: # Calculate node betweenness centrality as a measure of importance
print('Calculating betweenness...')
betweenness = networkx.betweenness_centrality(G, endpoints=False)
print(betweenness)
```

0.36 Crawling and analysing Wikiperdia graph: plotting the graph

```
[ ]: print('Drawing graph...')
# Sigmoid function to make the colors (a little) more appealing
squash = lambda x : 1 / (1 + 0.5**(20*(x-0.1)))
colors = [(0, 0, squash(betweenness[n])) for n in G.nodes()]
labels = dict((n, d['title']) for n, d in G.nodes(data=True))
positions = networkx.spring_layout(G)
networkx.draw(G, positions, node_color=colors, edge_color='#AEAEAE')
# Draw the labels manually to make them appear above the nodes
for k, v in positions.items():
    plt.text(v[0], v[1]+0.025, s=labels[k],
             horizontalalignment='center', size=8)
plt.show()
```

0.37 Plotting graph con Gephi

- un tool open-source molto popolare per la visualizzazione di grafi e reti che presenta funzionalità più avanzate rispetto a **NetworkX**
- **Layout**
 - **ForceAtlas**: riduce la distanza tra nodi con molte connessioni e la aumenta tra quelli con poche connessioni
- **Colori**
 - possono essere associati a diverse caratteristiche dei nodi
 - possono indicare il rank del nodo/dell'arco rispetto a qualche statistica
- **Filtri**
 - possibilità di effettuare uno **zoom** su parti deverse del grafo
 - giant component
 - ego network
 - comunità di nodi
 - attributi particolari

```
[ ]: networkx.write_gexf(G, "Wikipedia.gexf")
```

- possibile esportare un grafo da **NetworkX** nel formato **GEXF** (Graph Exchange XML Format)
- è un formato interpretato da **Gephi**

0.38 Astrazione delle transazioni di Bitcoin: il DataSet

0.39 Astrazione delle transazioni di Bitcoin: il DataSet

0.40 Astrazioni effettuate nel dataset

I principali elementi di una transazione **Bitcoin** sono i seguenti

hash: identificatore della transazione

block_id: identificatore del blocco che contiene la transazione

version

size

lock_time: tempo prima del quale una transazione non può essere inserita in un blocco

vin_sz

vout_sz

inputs

outputs

0.41 Astrazioni effettuate nel dataset

I campi eliminati nel nostro dataset sono i seguenti:

version

size

lock_time

vin_sz

vout_sz

- nel dataset, tutti i campi hash sono stati sostituiti con un identificatore numerico id
- questo ha permesso di ridurre la dimensione del dataset
- inoltre le chiavi pubbliche e le corrispondenti firme sono state sostituite da identificatori

0.42 Analisi richieste

- controllare se esistono blocchi o transazioni invalide nel dataset ed eventualmente rimuoverli/e
- calcolare le seguenti statistiche
 - distribuzione dei blocchi: numero di transazioni per ogni blocco, considerando l'intero periodo di tempo e per mese
 - distribuzione delle fee spese in ogni transazione nell'intero periodo
 - quanti UTXO esistono, al momento del mining dell'ultimo blocco del data set?
 - quale è l'UTXO a cui è associato il valore più alto?
 - trovare tutte le transazioni generate tra due date
- proporre almeno una analisi non compresa tra le precedenti

0.43 Analisi di Bitcoin: il DataFrame delle transazioni

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

#costruzione dei DataFrames

Transactions = pd.read_csv('Dataset/Transactions.csv', names=['tx_id', 'blk_id'])
Inputs = pd.read_csv('DataSet/Inputs.csv', names=['in_id', 'tx_id', 'sig_id', '
↳ 'output_id'])
```

```
Outputs = pd.read_csv('DataSet/Outputs.csv', names=['output_id', 'tx_id', 'pk_id', 'value'])

print(Transactions.head())
print(Transactions.tail())
```

	tx_id	blk_id
0	1	0
1	2	1
2	3	2
3	4	3
4	5	4

	tx_id	blk_id
216621	216622	100016
216622	216623	100016
216623	216624	100016
216624	216625	100016
216625	216626	100017

- primo passo: osservazione del **DataSet**
- una sola transazione per blocco, nei primi blocchi del **DataSet**
 - si tratta di **Coinbase**
 - transazioni di ricompensa al miner che ha risolto la **Proof of Work**
 - probabilmente minate da **Nakamoto** stesso
- gli ultimi blocchi includono più transazioni
- numero ultima transazione >> numero dell'ultimo blocco
 - diverse blocchi contengono molte transazioni
- **nota bene:** lo stesso nome **output_id** come identificatore di un output e come riferimento in un campo input.
 - questo ci consentirà di effettuare il **merge** dei due dataset

0.44 Analisi di Bitcoin: il DataFrame degli Inputs

```
[2]: print(Inputs.head())
      print(Inputs.tail())
```

	in_id	tx_id	sig_id	output_id
0	1	1	0	-1
1	2	2	0	-1
2	3	3	0	-1
3	4	4	0	-1
4	5	5	0	-1

	in_id	tx_id	sig_id	output_id
292422	292423	216622	95854	258874

292423	292424	216623	161394	247315
292424	292425	216624	173913	263227
292425	292426	216625	174188	263611
292426	292427	216626	0	-1

- primo passo: osservazione del **DataSet**
- nelle prime righe l'**output_id** è sempre uguale a **-1**
 - questo conferma che si tratta di transazioni **Coinbase**: non c'è un puntatore a una transazione precedente
 - i bitcoin sono stati creati dal sistema, non provengono da una transazione precedente
 - gli **in_id** corrispondono ai **tx_id**
 - * un solo input dummy per ogni transazione, non significativo (non punta a un output precedente)
- nelle ultime righe: il numero di **in_id** > **tx_id**
 - esistono transazioni con più di un input, molte transazioni con un solo input

0.45 Analisi di Bitcoin: il DataFrame degli Outputs

```
[3]: print(Outputs.head())
      print(Outputs.tail())
```

	output_id	tx_id	pk_id	value
0	1	1	1	5000000000
1	2	2	2	5000000000
2	3	3	3	5000000000
3	4	4	4	5000000000
4	5	5	5	5000000000

	output_id	tx_id	pk_id	value
264305	264306	216623	174700	1000000
264306	264307	216623	174701	299000000
264307	264308	216624	167815	1000000
264308	264309	216625	174695	1200000
264309	264310	216626	174701	5000000000

- primo passo: osservazione del **DataSet**
- le prime transazioni contengono un solo output
 - l'output corrisponde alla ricompensa data ai miners
 - inizialmente era di 5000000000 Satoshi = 50 BTC, poi dimezzata ogni 4 anni
- nelle ultime righe il numero di **output_id** > **tx_id**
 - esistono transazioni con output multipli
 - ad esempio la transazione 216623 ha due output diversi

0.46 Controlli di validità delle transazioni: quali?

- Input/output che fanno riferimento ad un **tx_id** non contenuto in nessun blocco
- Input che fanno riferimento ad un **output_id** non esistente
- Transazioni con **pk_id**/**sign_id** negativi

- Output con campo “amount” negativo
- Transazioni in double spending
- Reward per il miner minore di 50 BTC
- Si cercano di spendere dei bitcoin appartenenti ad un indirizzo diverso dal proprio
 - la chiave pubblica non è consistente con la firma
 - nel nostro caso, l’identificatore nell’output non corrisponde con quello nell’input

0.47 Controlli di validità delle transazioni: corrispondenza chiave pubblica-firma

- le chiavi pubbliche e le firme sono astratte nel dataset con degli identificatori
- la corrispondenza tra chiave pubblica e signature è verificata in **Bitcoin** da uno script **FORTH**
- nel nostro caso occorre verificare solamente la corrispondenza tra gli identificatori

```
[4]: #ricerca di record collegati tra input e output che non hanno i campi sig_id e
      ↪pk_id uguali

print("ricerca corrispondenza errate tra pk_id e sig_id")
print()

InputOutput = pd.merge(Outputs, Inputs, on="output_id")
print(InputOutput.head())
print()

Invalid_pk_sig = InputOutput.loc[InputOutput ['sig_id'] != InputOutput.
      ↪['pk_id']]

print("Transazioni inconsistenti trovate =", len(Invalid_pk_sig.index))
print()
print(Invalid_pk_sig)
```

ricerca corrispondenza errate tra pk_id e sig_id

	output_id	tx_id_x	pk_id	value	in_id	tx_id_y	sig_id
0	10	10	10	5000000000	172	172	10
1	172	172	172	1000000000	219200	156787	172
2	173	172	10	4000000000	184	184	10
3	186	184	10	3000000000	186	186	10
4	188	186	186	1000000000	228	228	186

Transazioni inconsistenti trovate = 2

	output_id	tx_id_x	pk_id	value	in_id	tx_id_y	sig_id
1678	16121	16081	16020	5000000000	194075	138278	139250
83064	137338	116208	113300	5000000000	157972	116411	-1

0.48 Controlli di validità delle transazioni: corrispondenza chiave pubblica-firma

- **merge**: una **join** su cui si può specificare la colonna su cui fondere i dataframe
 - per default calcola un **inner join**
 - calcola l'intersezione dei due **DataFrame**
- per individuare le transazioni invalide, si sfrutta uno dei tanti modi di utilizzare la funzione **Python loc**
 - si crea un **row indexer**, un Serie di valori **Booleani**

```
InputOutput.loc[InputOutput ['sig_id'] != InputOutput ['pk_id']]
```

- la Serie è creata confrontando valori corrispondenti delle due serie, corrispondenti a due colonne del **DataFrame**

```
InputOutput ['sig_id']  
InputOutput ['pk_id']
```

- si passa il **row indexer** di valori booleani alla funzione **loc**
- la **Series** di valori booleani deve essere **allineata** con le righe del **DataFrame**
- solo le righe in cui il valore corrispondente in **row indexer** è **True**, vengono selezionate
- viene costruito un nuovo **DataFrame** con le righe selezionate

0.49 Calcolo valore totale UTXO: individuazione output non spesi

- quale è il valore degli UTXO, al momento del mining dell'ultimo blocco del data set?
- **UTXO** valori non ancora spesi, stanno nell'output di una transazione, ma non sono riferiti dall'input di nessuna altra transazione

```
[5]: print("Calcolo valore totale in UTXO")  
  
# ricerca gli Output non spesi  
# ovvero gli Output che non hanno una corrispondenza in Inputs  
# essi rappresentano gli UTXO (Unspent Transaction Outputs)  
  
temp = (~Outputs['output_id'].isin(Inputs['output_id']))  
print(type(temp))  
print(temp.describe())  
print(temp.head())
```

Calcolo valore totale in UTXO

```
<class 'pandas.core.series.Series'>  
count      264310  
unique         2  
top         False  
freq       192404  
Name: output_id, dtype: object
```

```

0    True
1    True
2    True
3    True
4    True
Name: output_id, dtype: bool

```

- si utilizza la funzione: `python S.isin(V)`
- controlla se gli elementi della series **S** sono contenuti nella serie **V**
- restituisce una **Series di valori booleani** che indica
 - per ogni elemento della serie **S**, se quell'elemento è contenuto nella sequenza **V**
 - tale serie di valori booleani verrà poi passata alla funzione **loc**

0.50 Calcolo valore totale UTXO: individuazione output non spesi

```

[6]: print("CALCOLO TOTALE UTXO")

# ricerca gli Output non spesi
# ovvero gli Output che non hanno una corrispondenza in Inputs
# essi rappresentano gli UTXO (Unspent Transaction Outputs)

UTXO = Outputs.loc[temp]
print(UTXO)
print(UTXO.value)

```

```

CALCOLO TOTALE UTXO

```

	output_id	tx_id	pk_id	value
0	1	1	1	5000000000
1	2	2	2	5000000000
2	3	3	3	5000000000
3	4	4	4	5000000000
4	5	5	5	5000000000
...	...	...	...	...
264305	264306	216623	174700	1000000
264306	264307	216623	174701	299000000
264307	264308	216624	167815	1000000
264308	264309	216625	174695	1200000
264309	264310	216626	174701	5000000000

```

[71906 rows x 4 columns]
0    5000000000
1    5000000000
2    5000000000
3    5000000000
4    5000000000
...

```

```

264305      1000000
264306     299000000
264307      1000000
264308      1200000
264309     5000000000
Name: value, Length: 71906, dtype: int64

```

0.51 Calcolo dell'UTXO massimo

```

[14]: #ricerca del valore massimo

MaxUTXO = UTXO.value.max()
print("Massimo UTXO =", MaxUTXO )
SumUTXO = UTXO.value.sum()
print("Somma totale UTXO =", SumUTXO )
#cerco la somma di tutti i valori

MaxUTXO_tx = UTXO.loc[UTXO['value'] == MaxUTXO] #cerco la riga con valore
↳ massimo
MaxUTXO_tx = pd.merge(MaxUTXO_tx, Transactions, on=['tx_id'])
print(MaxUTXO_tx)

```

```

Massimo UTXO = 9000000000000
Somma totale UTXO = 500140414333354
   output_id  tx_id  pk_id      value  blk_id
0      170430  140479  138895  9000000000000    90532

```

0.52 Analisi delle transazioni: calcolo delle fee

- calcolare la distribuzione delle fee spese in ogni transazione nell'intero periodo contenuto nel **DataSet**
 - considerare tutte le transazioni, a parte le **CoinBase**
- calcolo delle fees per ogni transazione

$$\$(input) - (output)\$$$
 dove **input** e **output** sono i valori in input e output della transazione
- calcoliamo la somma dei **valori in input** e quella dei **valori in output** separatamente
 - somma dei valori in output di una transazione
 - * group_by su Output in base a tx_id
 - somma dei valori in input di una transazione
 - * i valori in input di una transazione sono contenuti nel precedente **UTXO**
 - * occorre fare un merge tra Input e Output su output_id
 - * attenzione a non confondere i tx_id in Output e quello in Input quando si fa la merge!
 - per il calcolo delle fee, interessa tx_id di Input !

0.53 Analisi delle transazioni: calcolo valore in output di ogni transazione

```
[ ]: # elimino la colonna relativa allo script di firma, inutile

Out=Outputs.drop(['pk_id'], axis="columns")

# raggruppo gli output per transazione
# ogni gruppo comprende gli output di una transazione
# aggrego, sommando per calcolare la somma degli output uscenti di ogni
↳ transazione

OutTrans = Out.groupby('tx_id')['value'].sum().reset_index()
OutTrans
```

- il campo **pk_id** non è utile per questa analisi, riguarda lo script per la verifica della firma
 - eliminiamo la colonna corrispondente dal **DataSet**
- **drop** rimuove una colonna o una riga specifica
 - **axis** specifica se si vuole eliminare una colonna o una riga
 - * **axis="columns"** - colonna
 - * **axis="index"** - riga
 - la colonna viene eliminata nel nuovo dataset creato, non in quello originale

0.54 Analisi delle transazioni: calcolo valore in input di ogni transazione

- il problema della determinazione dei valori in input è più complesso, perchè
 - il valore di un input di una transazione si trova in un **Unspent Transaction Output (UTXO)** di una transazione precedente
 - **group_by** non è sufficiente, ora serve collegare due diverse transazioni, quella che genera l'**UTXO**, e quella che lo spende
 - * si utilizza una **merge**

```
[ ]: Inp = Inputs.drop(['sig_id'] , axis=1)
Out = Outputs.drop(['tx_id', 'pk_id'], axis=1)
InpValues = pd.merge(Inp, Out, on='output_id')
InpValues
```

- **drop**: elimina i campi non importanti per questa analisi
 - nota bene: si elimina **tx_id** dal **DataSet** degli **Output**, perchè non interessa l'indice della transazione che produce l'**UTXO**, ma di quella che consuma gli output della transazione precedente
- **merge**
 - come una **inner join** sui due **DataFrame**
 - combina le righe con valore comune dell'attributo **output_id**
 - ogni input viene collegato all'output di una transazione precedente, di cui spende il valore
- analizziamo la riga 192403

292419 216618 264284 13836000000

* la transazione `216618` ha un input di indice `292419` che "consuma"
 * non interessa l'indice della transazione precedente, solo il valore in output che viene speso

0.55 Analisi delle transazioni: Coinbase

- la procedura precedente non considera le **Coinbase**
- cosa è una **Coinbase**?
 - la prima transazione in ogni blocco della blockchain
 - utilizzata per inviare la ricompensa al miner che ha minato quel blocco
- il formato è lo stesso delle altre transazioni eccetto che
 - ha esattamente un input
 - quell'input non punta ad un **UXTO** precedente e non contiene un dato significativo
 - può contenere messaggi arbitrari
 - il messaggio di **Nakamoto** nella prima Coinbase: *"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks"*
 - non ha fee, perchè le transazioni sono inserite nel blocco dal miner stesso
- gli output della transazione
 - possono essere in numero arbitrario
 - uno o più indirizzi del miner
 - la somma dei valori in output non può superare la somma della ricompensa data al miner con le fee di tutte le transazioni di quel blocco.

0.56 Analisi delle transazioni: Coinbase

- Nel nostro DataSet
 - le **Coinbase** sono transazioni che contengono un solo input
 - quell'input ha valore **-1**
 - il `sig_id` vale **0**
- le coinbase non sono incluse nel risultato del merge, perchè non c'è corrispondenza con un precedente `output_id`
- verifichiamolo!

```
[ ]: print(InpValues.loc[InpValues["output_id"]==-1])
```

0.57 Analisi delle transazioni: calcolo valore in input di ogni transazione

```
[ ]: InpValues.rename(columns={'in_id':'in_id', 'tx_id':'tx_id', 'output_id':
    ↳'utxo_id', 'value':'value_to_be_spent'}, inplace=True)
InpTrans = InpValues.groupby('tx_id')['value_to_be_spent'].sum().reset_index()
InpTrans
```

0.58 Analisi delle transazioni: calcolo delle fee

```
[ ]: #raggruppamento per indice della transazione
#unisco i Dataframe per avere, per ogni transazione, il valore speso e il
    ↳valore uscente

Tx = pd.merge(InpTrans, OutTrans, on='tx_id')
```

```

#aggiungo la colonna delle fees

Tx['fees'] = Tx['value_to_be_spent'] - Tx['value']

TxGreter0 = Tx.loc[Tx['fees'] > 0].sort_values('fees')
TxLessEq0 = Tx.loc[Tx['fees'] == 0]
TxLessLess0 = Tx.loc[Tx['fees'] < 0].sort_values('fees')

TxGreter0 , TxLessEq0, TxLessLess0

```

- esiste una transazione con fee negativa: un errore nel Dataset

0.59 Analisi delle transazioni: calcolo delle fee

```

[ ]: #plotting della distribuzione delle fees con valore maggiore di 0
Tx_Correct=Tx.loc[Tx['fees']>=0]
print(Tx_Correct)
X = list(Tx_Correct['tx_id'])
mat.plot(X,Tx_Correct['fees'])
mat.xlabel('Id transazioni che includono le fees', fontsize=13)
mat.ylabel('Valore fees', fontsize=13)
mat.yscale('log')
mat.show()

```

```

[ ]:

```